Veloce AgenticOS

System Architecture and Production-Readiness Plan

STATUS	OWNER	LAST UPDATED
Draft - Proposed Baseline	AgentOS Engineering	October 14, 2026

Authors	Aryaman
Reviewers	Engineering and security review pending
Related docs	README.md; docs/production-topology-2026-09-25.md; docs/public-beta-operating-contract-2026-09-25.md; docs/gcp-client-runtime-runbook.md; docs/production-readiness-execution-board.md

Scope	Current architecture, production-readiness gaps, verification strategy, and gated rollout sequence.

1. Abstract

AgentOS is a governed AI workspace with two distinct operating surfaces: a local operator environment and a tenant-facing cloud runtime. The proposed baseline keeps tenant identity, authorization, durable jobs, audit records, budgets, and artifacts explicit; provider calls run only through server-side workers. This design documents the existing shape, names evidence gaps, and defines the verification sequence required before any production-readiness claim.

The immediate workload is a human-reviewed client workflow: select approved sources, create a cited proposal, audit, or report draft, and preserve provenance. The system must fail closed on tenant authorization, approval, budget, and provider configuration. Hosted parity, exact retention, deployment identity, recovery, and end-to-end customer experience remain acceptance gates, not assumed guarantees.

2. Goals and Non-Goals

Goals	Non-goals
Prove a tenant-scoped, human-reviewed workflow from approved sources to a provenance-bearing draft.	Do not claim broad platform-market fit or proven willingness to pay.
Make local, Cloud Test, internal beta, and production configuration/build identity observable and comparable.	Do not merge the public entry surface with the authenticated tenant runtime.
Enforce server-side identity, tenant, approval, budget, and idempotency checks before external work.	Do not enable autonomous publishing or provider calls by default.
Establish verified retention, recovery, rollback, and release evidence before promotion.	Do not change production databases, IAM, retention, or deploy under this document alone.

3. Background and Problem Statement

The repository describes a local-first operator product and a separate tenant runtime. The public Vercel experience is an entry/editorial surface; it is not the authenticated workspace. The tenant runtime is hosted separately on Cloud Run and uses Supabase Auth/Postgres, asynchronous jobs, and server-side integrations. This split is intentional, but deployment drift has made it difficult to know which code and policies a user is exercising.

Recorded release evidence identifies material gaps: hosted build identity and source-import routes were absent from an earlier runtime; observed Cloud Test retention differed from the 45-day target; storage recovery adds a seven-day window; and the latest strict readiness run failed because the release-ledger record was missing while 38 categories were not evaluated. These are evidence snapshots, not proof that every environment is unchanged today. Governing invariant: no tenant data or external side effect crosses a boundary unless identity, tenant scope, policy, budget, approval, and audit state are checked server-side.

4. Proposed Architecture

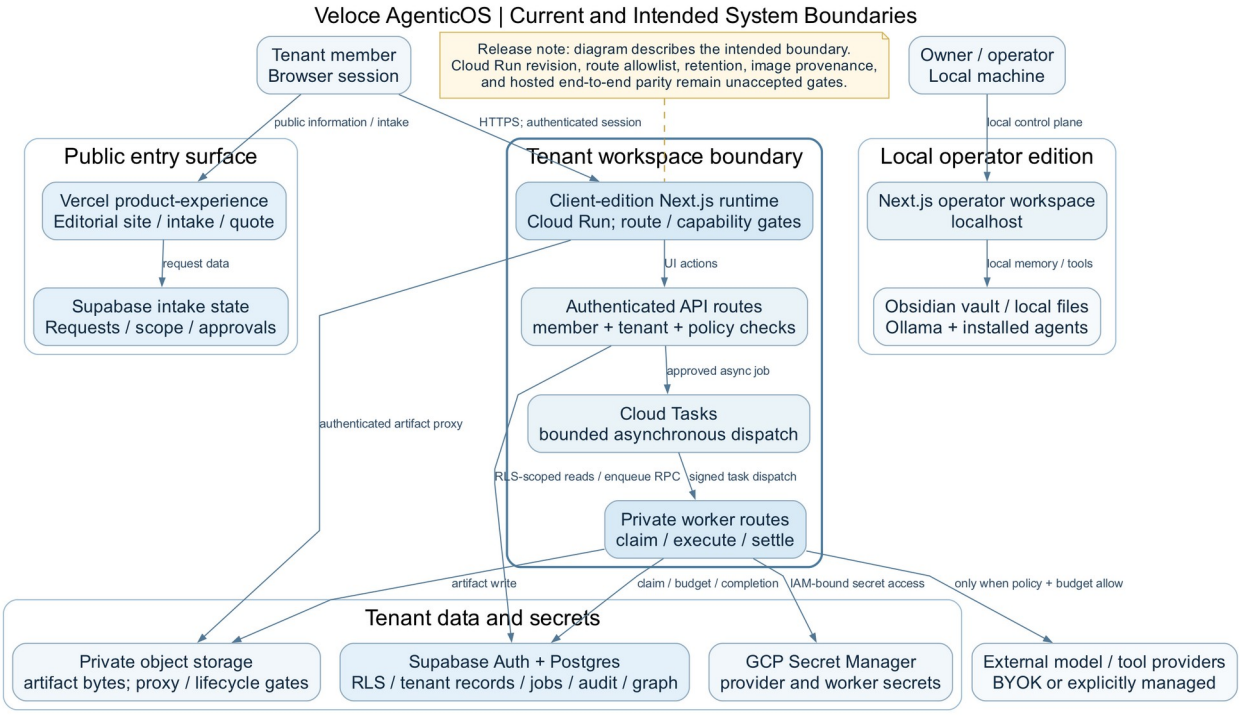


Figure 1. Proposed AgentOS system boundary and control/data flow. Hosted release gates shown as unaccepted.

Core components

Component	Responsibility	Primary storage	Failure behavior
Public product experience	Vercel-hosted public/editorial entry and acquisition surface; routes users toward the workspace.	Public content and web deployment; not tenant source-of-truth data.	Must not imply authenticated workspace availability when runtime is unavailable.
Local operator workspace	Local-first control and authoring environment for operator work, configuration, and development.	Local files and configured integrations; secrets outside source control.	Local failures must not silently change hosted tenant state.
Tenant client runtime	Cloud Run authenticated workspace/API; validates session, membership, capability, and policy.	Cloud Run revision/config plus Supabase-backed tenant state.	Fail closed on missing identity, membership, allowlist, policy, or required config.
Tenant control plane	Supabase Auth/Postgres stores memberships, requests, jobs, artifacts metadata, audit events, and budget ledger.	Postgres with RLS and server-side RPC boundaries.	Reject unauthorized tenant reads/writes; migrations require environment review and rollback tests.

Component	Responsibility	Primary storage	Failure behavior
Async worker and integrations	Cloud Tasks/worker executes approved jobs and calls providers using server-side secrets.	Queue, Secret Manager, object storage, provider APIs.	Bound attempts, timeouts, and spend; retries idempotent; uncertain outcomes auditable.

5. Request Lifecycle

a. User submits a capability request with tenant context, approved source IDs, output type, and idempotency key; source text is resolved server-side.

b. Runtime validates the Supabase session, membership, schema, capability allowlist, and environment identity. Mismatch returns a typed denial.

c. Runtime loads tenant policy, approval state, provider/model allowlists, budget, source ownership, and governing policy/config version.

d. For drafting, selected sources are retrieved and checked; output includes a source register. Unsupported claims remain reviewable, not asserted as facts.

e. Before asynchronous execution, API durably records an idempotent job and audit event. Enqueue failure must not cause untracked external effects.

f. Worker claims the job, rechecks policy and budget, records attempt/trace metadata, and executes only the allowed provider call with bounded retries.

g. Worker records terminal state, usage/settlement, artifact metadata, and provenance; user receives a reviewable result. Failure/cancellation remains auditable.

6. API and Data Contracts

Primary data contract

This design uses the existing client_runtime_jobs record as the durable execution contract. It is not a substitute for a versioned API schema; publish OpenAPI/JSON Schema for client-runtime routes before broad external integration.

Field	Type	Required	Description
job id	uuid	Yes	Stable identifier shared by worker, audit, and artifact references.
tenant_slug	text	Yes	Ownership scope derived/validated against authenticated membership, never trusted alone from client payload.
actor and capability	uuid + text	Yes	Authenticated requester and allowlisted execution intent; provider/model validated against server policy.
idempotency_key	text	Yes	Request deduplication within a tenant; unique with tenant_slug in current schema.

Field	Type	Required	Description
status / attempt_count	enum / integer	Yes	Queued, running, succeeded, failed, or cancelled; bounded attempts (schema permits 0–10).
input / output metadata	jsonb	Yes	Structured references and non-secret metadata; use IDs rather than duplicated source text where possible.
error_code / timestamps	text / timestamptz	Yes	Typed terminal reason and lifecycle timestamps; no credentials or raw sensitive payloads in logs.

Source selection should use tenant-scoped IDs and immutable/versioned references, not client-supplied document bodies. Artifact metadata should capture job, tenant, source IDs/versions, generation config, review state, and retention class.

Contract guarantees

Required contract guarantees:

a. Authorization and tenant scope are revalidated server-side before source access or execution.

b. One durable logical job per tenant/idempotency key; duplicates return or resume the same job.

c. Capture policy/config version, source references, model/provider, and actor before execution.

d. Job state is authoritative for execution; object storage is authoritative for bytes; database metadata links them.

The versioned route/schema contract is not yet consolidated. Publish it under

docs/api-contracts/client-runtime (proposed) and enforce compatibility with contract tests before release.

7. Consistency, Idempotency, and Replay

The control plane provides tenant-scoped idempotency and durable job state. Treat queue delivery as at-least-once: workers conditionally claim/transition jobs and never repeat a completed side effect. Provider timeouts are ambiguous outcomes; record them distinctly and reconcile before retrying non-idempotent actions. Snapshot configuration at enqueue and recheck authorization/budget at execution. Replay is a separately authorized and audited recovery action, not a duplicate-submit mechanism.

Scenario	Expected behavior	Reasoning
Duplicate or repeated request	Return the existing job/result for the same tenant and idempotency key; do not enqueue duplicate work.	The unique tenant/key constraint prevents duplicate logical execution.
Durable write or enqueue fails	Return a retryable error only when no job was committed; otherwise return the persisted job state.	Avoids untracked downstream side effects and ambiguous client retries.

Scenario	Expected behavior	Reasoning
Timeout or partial provider failure	Record an explicit unknown/failed state; reconcile before replaying any non-idempotent side effect.	Preserves budget accounting and auditability when provider completion is uncertain.
Policy/config changes mid-job	Use the captured execution config for deterministic interpretation, but recheck authorization and budget before side effects.	Prevents accidental policy drift while allowing security revocation to take effect.

8. Security and Privacy Considerations

a. Authenticate with Supabase Auth; authorize every read/write by tenant membership and capability. RLS and server-side functions are defense in depth; do not trust client-supplied tenant IDs.
b. Minimize retained source content and provider payloads. Prefer references and redacted metadata; never log API keys, access tokens, cookies, or full sensitive prompts by default.
c. Keep provider credentials in server-side Secret Manager/config. Browser bundles and client responses must contain no provider secrets; audit service-account scope.
d. Administrative import, migration, replay, deletion, and support access require environment scoping, least privilege, dry-run evidence, and immutable audit records.
e. 45 days per item is the stated customer-data target, not a verified system-wide guarantee. Test DB, artifacts, backups, logs, derived graph data, and recovery separately; billing exceptions need an approved schedule.

9. Operational Readiness

Signal	SLO or alert	Owner	Launch gate
Customer journey and tenant isolation	All required journey assertions pass; cross-tenant tests deny; build/config identity matches intended environment.	Product/engineering	Required before beta sign-off
Source import and graph integrity	Dry-run has zero unexplained omissions/duplicates; tenant-scoped import/search tests pass; provenance retained.	Data/engineering	Required before corpus import
AI quality and spend controls	Offline eval rubric passes; injection/grounding cases pass; budget/approval enforcement tested with provider mocked.	AI/product	Required before provider-enabled release

Signal	SLO or alert	Owner	Launch gate
Retention, storage, recovery	Per-item deletion/recovery tested across DB/object/derived data; 45-day target or approved exception; restore drill recorded.	Platform/security	Required before scaled customer data
Artifact integrity and release gate	Current build identity verified; release ledger exists; mandatory categories evaluated; rollback target exercised.	Release owner	Required before production

Rollout constraint: Begin with local/offline checks and dry-runs. Hosted checks stay read-only until target, build identity, and config are verified. Internal beta needs explicit environment approval, provider calls off unless separately authorized, tested rollback, and complete mandatory evidence. This document does not authorize production mutation or deployment.

10. Alternatives Considered

Alternative	Why it was considered	Why it was not selected
Collapse all surfaces into one Vercel app	Fewer URLs and simpler deployment.	The current split separates public entry from tenant runtime; collapsing before boundary proof increases blast radius.
Run provider calls in browser	Less worker complexity for a prototype.	Exposes credentials and weakens spend, policy, audit, and retry controls.
Replace Supabase/job stack now	Could produce a cleaner greenfield design.	Migration risk adds work without closing current evidence gaps.
Add graph/observability platform immediately	Could improve visualization or traces.	First prove correctness and limits with current foundations; add infrastructure only for a demonstrated gap.

11. Open Questions

a. Which Cloud Run URL/revision is canonical Cloud Test, and who owns approval for production-backed Supabase test tenants?
b. Is 45-day deletion absolute or a target with approved billing/security exceptions? Who is the named reviewer?
c. What isolation is required for the first paid cohort: shared DB with tenant RLS, separate schema/database, or separate cloud project?
d. Who owns on-call, incident response, restore drills, release review, and support? What SLOs will they accept?

12. Decision and Next Steps

Decision: adopt the existing split-surface architecture as the proposed baseline and close evidence gaps through one ordered release path. Do not add runtime infrastructure until tests show a concrete deficiency. M1 establishes a reproducible current-source artifact and build identity. M2 passes local customer-journey, tenant-isolation, and contract tests. M3 reconciles and dry-runs authorized source imports. M4 proves retention, recovery, provenance, offline AI quality, spend, and approval controls. M5 verifies hosted target/config/build, completes the strict readiness ledger, exercises rollback, and records go/no-go. Promotion is blocked while any mandatory category is unevaluated or release-ledger evidence is missing.

Milestone	Deliverable	Exit criteria
M1 - Artifact integrity	Build/package current source with supported runtime; expose and verify build identity and required route set.	Reproducible artifact; verifier passes; source commit and image digest recorded.
M2 - Customer journey	Exercise login, workspace, source selection, draft, review, artifact access, recovery, and cross-tenant denial locally.	Current-source Playwright journey and tenant-isolation tests pass without provider spend.
M3 - Source corpus	Reconcile authorized archive/files/media references; validate tenant-scoped graph import/search in dry-run.	Counts reconcile; omissions explained; replay/rollback tested; corpus scope confirmed.
M4 - Runtime and release controls	Prove retention/recovery, provenance, offline AI evals, provider allowlist, spend/approval, hosted identity, release ledger, rollback.	All mandatory evidence recorded; explicit reviewer sign-off and go/no-go; any missing category blocks promotion.
M5 - Promotion decision	After M4, perform approved hosted validation and release review.	All required gates pass; exact target/revision verified; tested rollback; explicit go/no-go recorded.